1. Aim

To design and analyse an OTP verification program using while and for loops, and to demonstrate break, continue, pass and the loop–else clause while ensuring that blank entries do not count as attempts.

2. Problem Explanation

A bank kiosk sends a one-time password (OTP) to a customer. The customer is allowed three non-blank attempts to enter the correct OTP. A blank entry must not reduce the number of attempts. If all three non-blank entries are incorrect, the account is locked for 30 minutes. The program must stop immediately when the correct OTP is entered.

3. Loop Design Using While

A while loop is suitable because the number of attempts depends on valid user input. The counter is increased only after a non-blank OTP is entered.

```
correct_otp = "4826"
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    entered = input("Enter OTP: ").strip()

    if entered == "":
        print("Blank entry. Please enter the OTP.")
        continue

    attempts += 1

    if entered == correct_otp:
        print("OTP verified successfully.")
        break
    else:
        print("Incorrect OTP.")
else:
    print("Account locked for 30 minutes.")
```

The continue statement skips the remaining statements for a blank entry and starts the next iteration. Since the counter is not increased, blank entries do not consume attempts. The break statement stops the loop when the OTP is correct. The loop else block executes only if the loop finishes normally without break.

4. Loop Design Using For with range(3)

A for loop with range(3) provides three valid attempt slots. An inner while loop repeats the input step until the customer enters a non-blank value.

```
correct_otp = "4826"

for attempt in range(3):
    while True:
        entered = input("Enter OTP: ").strip()
```

```
        if entered == "":
            print("Blank entry. This does not count.")
            continue

        break

    if entered == correct_otp:
        print("OTP verified successfully.")
        break
    else:
        print("Incorrect OTP.")
else:
    print("Account locked for 30 minutes.")
```

The inner while loop handles blank entries. The outer for loop advances only after a non-blank entry is received. If all three entries are incorrect, the for loop finishes normally and executes its else block.

5. Use of break, continue, pass and Loop–Else

Statement	Purpose
break	Terminates the loop immediately when the correct OTP is entered.
continue	Skips the remaining statements for a blank entry and asks again.
pass	Does nothing; it is used as a placeholder where a statement is required.
loop–else	Executes the lock message only when the loop finishes without break.

Example demonstrating pass:

```
entered = "4826"

if entered.isdigit():
    pass
else:
    print("OTP should contain digits.")
```

The pass statement does not verify the OTP or change the attempt count. It simply acts as an empty placeholder statement.

6. Test Cases and Expected Behaviour

Test	Entered values	Expected behaviour
1	Blank, 4826	Blank is ignored; OTP is verified on the first counted attempt.
2	1111, 2222, 4826	Two wrong attempts followed by successful verification.
3	Blank, 1111, blank, 2222, 3333	Only three non-blank entries count; account is locked.
4	4826	OTP is verified immediately; remaining attempts are not used.

Sample output for a blank entry followed by a correct OTP:

```
Enter OTP:
Blank entry. Please enter the OTP.
Enter OTP: 4826
OTP verified successfully.
```

Sample output after three incorrect non-blank entries:

```
Enter OTP: 1111
Incorrect OTP.
Enter OTP: 2222
Incorrect OTP.
Enter OTP: 3333
Incorrect OTP.
Account locked for 30 minutes.
```

7. Comparison of While and For Loops

While loop	**For loop with range(3)**
Repeats while the attempt condition is true.	Runs for a fixed three attempt slots.
The counter must be updated by the programmer.	The loop variable advances automatically.
Natural when the attempt limit may change dynamically.	Clear when the maximum number of attempts is fixed.
Blank entries can be retried without increasing the counter.	An inner while loop is used to retry blank entries.

A while loop is more natural when the number of attempts depends on valid input or may change during execution. If the counter is increased before checking for blank input, blank entries incorrectly consume attempts. If the counter is not increased after a wrong non-blank OTP, the loop may continue indefinitely.

8. Changed Condition – Two Extra Attempts

If the customer answers the security question correctly, two extra attempts are allowed. Therefore, the maximum becomes five. If the answer is incorrect, the limit remains three.

```
correct_otp = "4826"
security_answer = input("Answer security question: ").strip().lower()

if security_answer == "blue":
    max_attempts = 5
else:
    max_attempts = 3

attempts = 0

while attempts < max_attempts:
    entered = input("Enter OTP: ").strip()

    if entered == "":
        print("Blank entry. This does not count.")
        continue

    attempts += 1

    if entered == correct_otp:
        print("OTP verified successfully.")
        break
    else:
        print("Incorrect OTP.")
```

```
else:
    print("Account locked for 30 minutes.")
```

The word blue is only an example security answer for demonstration. A real system should verify security answers securely and should not store sensitive answers as plain text.

9. Alternative Concept – Recursion

The OTP verification process can also be implemented using a recursive function. The function calls itself for the next attempt until the OTP is correct or no attempts remain.

```
def verify_otp(correct_otp, attempts_left):
    if attempts_left == 0:
        print("Account locked for 30 minutes.")
        return

    entered = input("Enter OTP: ").strip()

    if entered == "":
        print("Blank entry. This does not count.")
        return verify_otp(correct_otp, attempts_left)

    if entered == correct_otp:
        print("OTP verified successfully.")
        return

    print("Incorrect OTP.")
    return verify_otp(correct_otp, attempts_left - 1)


verify_otp("4826", 3)
```

A blank entry calls the function again with the same number of attempts remaining. A wrong non-blank entry reduces attempts_left by one. A correct OTP ends the recursion.

10. Time and Space Complexity Analysis

Let a represent the maximum number of counted OTP attempts.

Method	Time complexity	Space complexity	Explanation
While loop	O(a)	O(1)	At most a valid attempts are processed; only a few variables are stored.
For loop	O(a)	O(1)	At most a attempt slots are processed; blank retries do not count.
Recursion	O(a)	O(a)	At most a wrong attempts cause recursive calls, using call-stack memory.

All three methods require O(a) time in the worst case. The loop versions use constant extra space, while recursion uses stack space proportional to the number of attempts.

11. Execution Evidence

After running the programs, add screenshots showing a blank entry, a successful OTP, and three incorrect entries leading to the account-lock message. The test-case table above records the expected behaviour.

- Screenshot of the while-loop execution.
- Screenshot of the for-loop execution.
- Screenshot proving that a blank entry does not reduce the attempt count.
- Screenshot showing the account-lock message after three incorrect non-blank attempts.

12. Conclusion

The OTP verification system was designed using while and for loops. The programs demonstrate break, continue, pass and loop–else. Blank entries are ignored and do not count as attempts. The while loop is flexible when the attempt limit changes, while the for loop is suitable for a fixed number of attempts. Recursion is another possible approach, but loops use less memory for this task.